First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

# I. Introduction

The goal of this lab project is the design and implementation of a half subtractor and full subtractor in Xilinx Verilog code with boolean arithmetic. Half and full subtractors are used in many circuits (and therefore digital systems), commonly used in applications such as binary subtraction and other arithmetic operations. The purpose of this project is to simulate real-world applications of boolean operations such as those implemented in a half and full subtractor within Verilog code; my aim is demonstrating an understanding of logical circuit design principles.

---

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

# II. Theoretical Preparation

## Designing a Binary Half-Subtractor

Table 1: Half-Subtractor

| **AB** | **D (difference)** | **B (borrow)** |
|---|---|---|
| 00 | 0 | 0 |
| 01 | 1 | 1 |
| 10 | 1 | 0 |
| 11 | 0 | 0 |

### Simplified Boolean Equations for Half-Subtractor

$D = f(A, B) = A'B + B'A = A \oplus B$

$B = f(A, B) = A'B$

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

# Designing a Binary Full-Subtractor

Table 2: Full-Subtractor

| **A** | **B** | **Bin** | **D** | **Bout** |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |

## Simplified Boolean Equations for Full-Subtractor

$D = f(A, B, Bin) = (A \oplus B) \oplus Bin$

$Bout = f(A, B, Bin) = A'Bin + A'B + BBin$

$= Bin(A \oplus B)' + A'B$

---

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

# III. Experimental Setup

## Verilog Code for Binary Half-Subtractor

HS.v

```
1  `timescale 1ns / 1ps
2  module HS(
3        i_bit1,
4        i_bit2,
5        o_difference,
6        o_borrow
7     );
8
9  input i_bit1; // A
10 input i_bit2; // B
11 output o_difference; // D
12 output o_borrow; // B (borrow)
13
14 assign o_difference = i_bit1 ^ i_bit2; // bitwise xor
15 assign o_borrow = ~i_bit1 & i_bit2; // bitwise and (A'B)
16
17 endmodule // half_subtracter
```

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

# Verilog Code for Binary Full-Subtractor

FS.v

```
1  `timescale 1ns / 1ps
2  module FS(
3        i_bit1,
4        i_bit2,
5        i_borrow,
6        o_difference,
7        o_borrow
8      );
9
10 input i_bit1; // A
11 input i_bit2; // B
12 input i_borrow; // Bin
13 output o_difference; // D
14 output o_borrow; // B (borrow)
15
16 assign o_difference = (i_bit1 ^ i_bit2) ^ i_borrow;
17 assign o_borrow = (i_borrow & ~(i_bit1 ^ i_bit2)) | (~i_bit1 & i_bit2) ;
18
19
20 endmodule // full_subtracter
21
```

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

# IV. Experimental Result

## Waveform for Binary Half-Subtractor


Code assigns input variables i_bit1 (A) and i_bit2 (B) to every possible binary combination of (A,B) for 10ns each:

- (0, 0),
- (0, 1),
- (1, 0),
- (1, 1)

The outputs o_difference (D) and o_borrow (B) exhibit values that are consistent with the truth table for a half-subtractor.

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

# Waveform for Binary Full-Subtractor


Code assigns input variables i_bit1 (A) and i_bit2 (B), i_borrow (Bin) to every possible binary combination of (A,B, Bin),

- (0, 0, 0),
- (0, 0, 1),
- (0, 1, 0),
- (0, 1, 1),
- (1, 0, 0),
- (1, 0, 1),
- (1, 1, 0),
- (1, 1, 1)

holding each signal combination for 10ns.

The outputs o_difference (D) and o_borrow (B) exhibit values that are consistent with the truth table for a full-subtractor.

---

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

# V. Conclusion

In conclusion, this project has implemented both a half subtractor and a full subtractor in Verilog code. The implementation and design were based on the boolean expressions derived from the truth tables of each circuit, with the code showcasing the correct values for each output variable given a set of inputs. Through this lab project, experience has been gained in utilizing the Xilinx ISE and setting up virtual machines using Xilinx software, as well as in Verilog code. Overall, this demonstrates an understanding of logical circuit design principles such as using truth tables, and deriving the simplest boolean expressions through the use of Karnaugh maps and boolean theorems.

---